

SE4-DMAD — Kursusnoter

Diskret Matematik, Algoritmer og Datastrukturer

Casper Holm Bach

Indhold

1	Logik	4
1.1	Logiske operatorer og præcedens	4
1.2	Sandhedstabeller	4
1.3	Logiske ækvivalenser	4
1.4	Kvantorer og indlejrede kvantorer	4
1.5	Eksamenstips og faldgruber	4
2	Bevismetoder	4
2.1	Direkte bevis	4
2.2	Kontrapositionsbevis	4
2.3	Modstridsbevis	4
2.4	Induktionsbevis	4
2.5	Eksamenstips og faldgruber	4
3	Relationer	4
3.1	Egenskaber	4
3.2	Ækvivalensrelationer	4
3.3	Partielle ordninger	4
3.4	Lukninger	4
3.5	Eksamenstips og faldgruber	4
4	Asymptotisk analyse	4
4.1	Hvad er en algoritme?	4
4.2	Analyse af en algoritme	5
4.3	Best, average og worst case	5
4.4	Definitioner	5
4.5	Sammenligning af voksehastigheder	8
4.6	Asymptotisk forhold via grænseværdier	9
4.7	Vækstrategierarki	10
4.8	Invarianter	10
4.9	Analyse af løkker i hånden	11
4.10	Eksamenstips og faldgruber	12
5	Rekursionsligninger	12
5.1	Divide and conquer	12
5.2	Rekursionsligning	13
5.3	Rekursionstræ-metoden	14
5.4	Master-sætningen (CLRS 4. udgave)	15
5.5	Matrixmultiplikation	18
5.6	Eksamenstips og faldgruber	21
6	Grafgennemløb	21
6.1	Repræsentationer	21
6.2	Bredde-først søgning (BFS)	21
6.3	Dybde-først søgning (DFS)	21
6.4	Topologisk sortering	21
6.5	Eksamenstips og faldgruber	21
7	Sortering	22
7.1	Sammenligningsbaserede sorteringer	22
7.2	Lineærtids-sorteringer	22
7.3	Sammenligningstabel	22
7.4	Eksamenstips og faldgruber	22
8	Minimum udspændende træ	22
8.1	Snit-egenskaben (cut property)	22
8.2	Kruskals algoritme	22
8.3	Prims algoritme	22

8.4	Eksamenstips og faldgruber	22
9	Disjunkte mængder (Union-Find)	22
9.1	Operationer	22
9.2	Union by rank	22
9.3	Path compression (stikomprimering)	22
9.4	Eksamenstips og faldgruber	22
10	Grådige algoritmer og Huffman-kodning	22
10.1	Det grådige princip	22
10.2	Huffman-kodning	22
10.3	Eksamenstips og faldgruber	22
11	Søgetræer	22
11.1	Binære søgetræer	22
11.2	Rød-sorter træer	23
11.3	Indsættelse og rotationer	23
11.4	Udvidede (augmented) træer	23
11.5	Eksamenstips og faldgruber	23
12	Heaps og prioritetskøer	23
12.1	Heap-egenskaben og array-repræsentation	23
12.2	Opbygning af en heap	23
12.3	Operationer	23
12.4	Eksamenstips og faldgruber	23
13	Dynamisk programmering	23
13.1	Principper	23
13.2	Gennemregnede eksempler	23
13.3	Køretids- og pladsanalyse	23
13.4	Eksamenstips og faldgruber	23
14	Hashing og ordbøger (dictionaries)	23
14.1	Kædning (chaining)	23
14.2	Åben adressering	23
14.3	Fyldningsgrad (load factor) og ydeevne	23
14.4	Eksamenstips og faldgruber	23
15	Korteste veje	23
15.1	Relaksering (relaxation)	23
15.2	Dijkstras algoritme	24
15.3	Bellman-Ford	24
15.4	Korteste veje i en DAG	24
15.5	Hvilken algoritme skal bruges	24
15.6	Eksamenstips og faldgruber	24

1 Logik

1.1 Logiske operatorer og præcedens

Noter tilføjes ($\neg \wedge \vee \Rightarrow \Leftrightarrow \oplus$, præcedenshierarki).

1.2 Sandhedstabeller

Noter tilføjes ($\#rækker = 2^{\text{antal variable}}$).

1.3 Logiske ækvivalenser

Noter tilføjes (f.eks. $p \Rightarrow q \equiv \neg p \vee q$, De Morgan; tautologi / modstrid / kontingens).

1.4 Kvantorer og indlejrede kvantorer

Noter tilføjes ($\forall, \exists$; rækkefølgen betyder noget; negation af kvantificerede udsagn).

1.5 Eksamenstips og faldgruber

Noter tilføjes.

2 Bevismetoder

2.1 Direkte bevis

Noter tilføjes.

2.2 Kontrapositionsbevis

Noter tilføjes.

2.3 Modstridsbevis

Noter tilføjes.

2.4 Induktionsbevis

Noter tilføjes (basis + induktionsskridt; genkende et gyldigt vs. ugyldigt bevis).

2.5 Eksamenstips og faldgruber

Noter tilføjes.

3 Relationer

3.1 Egenskaber

Noter tilføjes (refleksiv, symmetrisk, antisymmetrisk, transitiv).

3.2 Ækvivalensrelationer

Noter tilføjes (refleksiv + symmetrisk + transitiv; ækvivalensklasser).

3.3 Partielle ordninger

Noter tilføjes (refleksiv + antisymmetrisk + transitiv).

3.4 Lukninger

Noter tilføjes (refleksiv / symmetrisk / transitiv lukning).

3.5 Eksamenstips og faldgruber

Noter tilføjes.

4 Asymptotisk analyse

4.1 Hvad er en algoritme?

En algoritme er en betegnelse for en trinvis opskrift for hvordan en computer kan løse en given opgave eller problem. Den tager imod et input, bearbejder det, og returnerer et output, som er resultatet af algoritmen.

Mindstekravet for algoritmer er at de kan betegnes som **korrekte**:

- Algoritmen er ikke en uendelig løkke, og den stopper for inputs.
- Algoritmen giver et korrekt svar på det problem vi forsøger at løse med den.

En korrekt algoritme kan derudover have forskellige kvaliteter:

- Hastighed
- Pladsforbrug (RAM / Memory)
- Komplexitet af implementation
- Ekstra problemspecifikke egenskaber

4.2 Analyse af en algoritme

Vi er ofte interesseret i at måle disse kvaliteter af en algoritme. For at kunne analysere en algoritme skal man bruge følgende:

- Model af problem
 - Definer input og output.
- Model af maskine
 - Hvilket “type” computer kører algoritmen på? I dette kursus bruges RAM-modellen, hvor grundlæggende operationer tager konstant tid.
- Mål for kvalitet
 - I dette kursus fokuserer vi på at analysere tidsforbrug.
- Matematiske værktøjer
 - Asymptotisk notation til at beskrive køretidsvækst.
 - Invarianter der skal være sande under hele kørslen.
 - Induktion som bevisteknik.
 - Rekursionsligninger der beskriver køretid for rekursive algoritmer.

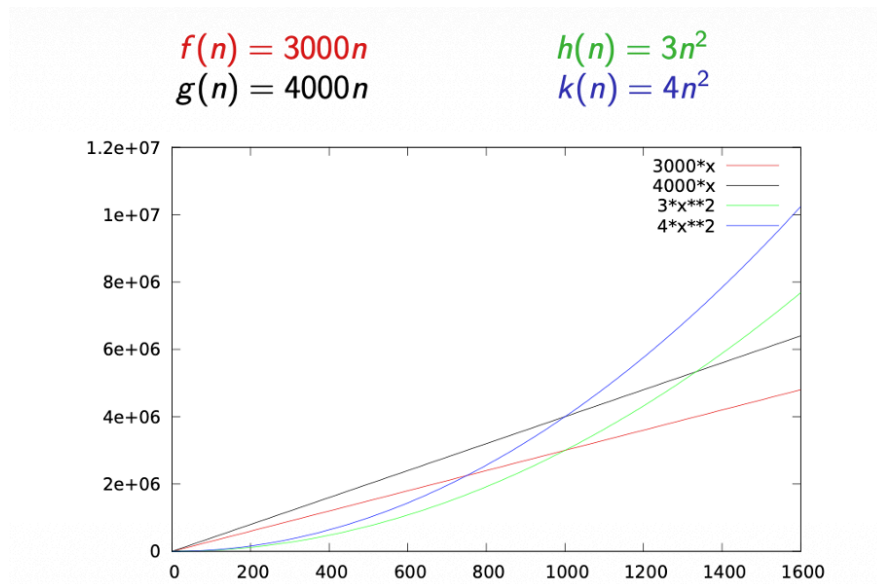
4.3 Best, average og worst case

Vi kan generelt måle en algoritmes hastighed i enten best, average eller worst case. Dog giver det bedst mening at måle i **worst case**-køretid, da dette giver os en garanti for hvad vi kan forvente. En worst case er ofte en voksende funktion.

4.4 Definitioner

Før vi begynder at fremstille en algoritme der kan løse vores problem, vil vi gerne kende voksehastigheden af de forskellige algoritmer så vi kan vælge den hurtigste. Det gør vi i det her kursus med RAM-modellen, som er bevist at afspejle rigtig køretid ret godt.

Når vi sammenligner forskellige voksehastigheder, er konstanter der bliver ganget på ligegyldige. Det vil altid være ledet der indebærer voksehastigheden (det led hvor n befinder sig), som dominerer.



Figur 1: På grafen ser vi at funktionerne $h(n)$ og $k(n)$ altid vil overhale $f(n)$ og $g(n)$, selvom deres konstant er betydelig mindre.

For at simplificere analysen af algoritmer bruger vi asymptotisk notation. Ved asymptotisk notation betragter vi alle konstanter der bliver ganget på for ens (c).

$$\{c \cdot f(n) \mid \text{alle } c > 0\}$$

Figur 2: Det vil sige alle algoritmer med samme voksehastighed bliver betragtet som lige gode.

Vi kan nu bruge følgende relationer til at sammenligne voksehastigheder, som hver især har en betegnelse:

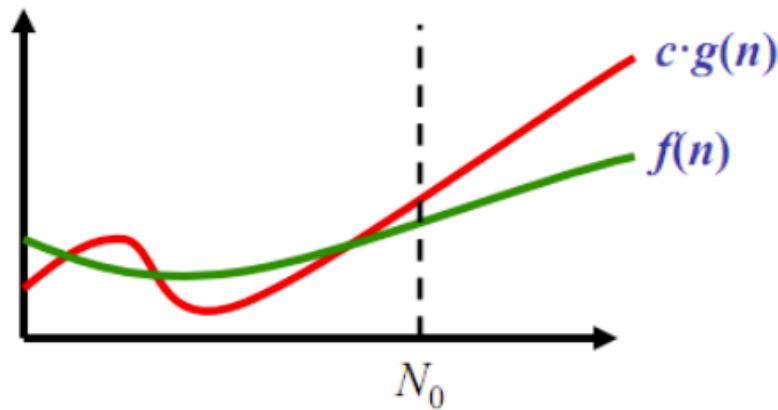
Relation	Betegnelse	Udtales
$\leq$	O	O
$\geq$	Ω	Omega
$=$	Θ	Theta
$<$	o	Lille o
$>$	ω	Lille omega

4.4.1 O (øvre grænse)

$$f(n) = O(g(n))$$

Hvis der findes en $c > 0$ og grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) \leq c \cdot g(n)$$



Figur 3: $g(n)$ sætter loftet for hvor hurtigt $f(n)$ må vokse.

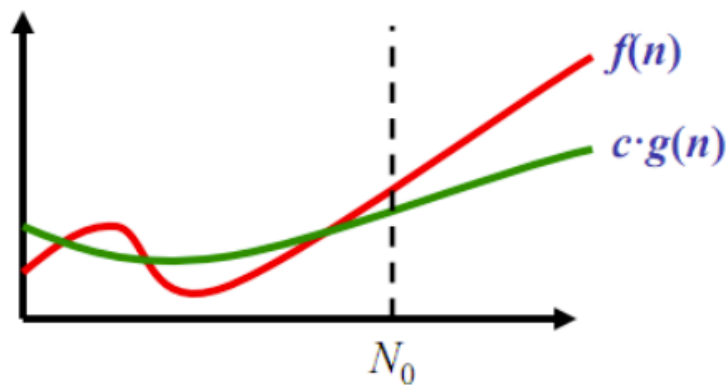
Det betyder altså at $g(n)$ sætter loftet for, hvor hurtigt $f(n)$ må vokse. $f(n)$ må gerne vokse samme hastighed, som $g(n)$. Der er ikke noget loft for hvor meget hurtigere $g(n)$ må vokse end $f(n)$.

4.4.2 Ω (nedre grænse)

$$f(n) = \Omega(g(n))$$

Hvis der findes en $c > 0$ og grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) \geq c \cdot g(n)$$



Figur 4: $g(n)$ sætter gulvet for hvor hurtigt $f(n)$ skal vokse.

Det betyder altså at $g(n)$ sætter gulvet for, hvor hurtigt $f(n)$ skal vokse. $f(n)$ må gerne vokse samme hastighed, som $g(n)$. Der er ikke noget loft for hvor meget hurtigere $f(n)$ må vokse end $g(n)$.

4.4.3 Θ

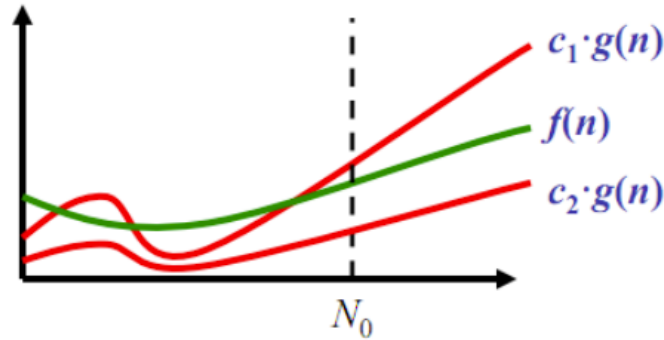
$$f(n) = \Theta(g(n))$$

Hvis der findes konstanter $c_1, c_2 > 0$ og en grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) = O(g(n)) \quad \text{og} \quad f(n) = \Omega(g(n))$$

Som også kan beskrives:

$$c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$$



Figur 5: f skal altid ligge mellem de to funktioner med hver sin konstant.

Det betyder at funktion f skal altid ligge mellem de to funktion g , der har hver sin konstant, efter grænsen N_0 .

4.4.4 o (streng øvre grænse)

$$f(n) = o(g(n))$$

Hvis der for alle $c > 0$ findes en grænse N_0 , så der for alle n efterfølgende ($n \geq N_0$) gælder:

$$f(n) \leq c \cdot g(n)$$

Som også kan fortolkes som:

$$f(n) < c \cdot g(n)$$

Fungerer altså ligesom store o , men f må ikke længere vokse samme hastighed som g .

4.4.5 ω (streng nedre grænse)

$$f(n) = \omega(g(n))$$

Hvis der for alle $c > 0$ findes en grænse N_0 , så der for alle n efterfølgende ($n \geq N_0$) gælder:

$$f(n) \geq c \cdot g(n)$$

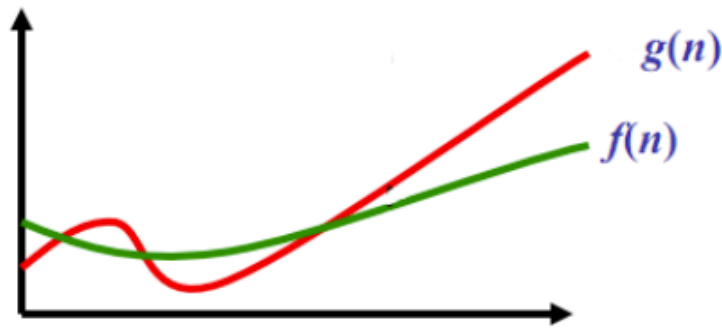
Som også kan fortolkes som:

$$f(n) > c \cdot g(n)$$

Fungerer altså ligesom store ω , men f må ikke længere vokse samme hastighed som g .

4.5 Sammenligning af voksehastigheder

For at sammenligne to algoritmers køretid skal vi sammenligne to funktioner, for at se hvordan hver algoritme vokser når inputtet n stiger.



Figur 6: Som ses på grafen overstiger $g(n)$ altid $f(n)$ på et tidspunkt, og derfor har $g(n)$ altså en større voksehastighed end $f(n)$.

4.6 Asymptotisk forhold via grænseværdier

Har man to funktioner, $f(n)$ og $g(n)$, kan man finde det asymptotiske forhold mellem dem ved at bruge følgende to sætninger, uden at kende c og N_0 :

Sætning 1: Hvis

$$\frac{f(n)}{g(n)} \rightarrow k > 0 \quad (\text{en positiv konstant}) \quad \text{når } n \rightarrow \infty$$

så gælder

$$f(n) = \Theta(g(n))$$

Det vil sige, at hvis forholdet mellem f og g konvergerer mod et fast positivt tal, betyder det at de to funktioner vokser lige hurtigt.

Eksempel:

$$f(n) = 20n^2 + 17n + 312$$

$$g(n) = n^2$$

$$\frac{f(n)}{g(n)} = \frac{20n^2 + 17n + 312}{n^2} = 20 + \frac{17}{n} + \frac{312}{n^2} \xrightarrow{n \rightarrow \infty} \frac{20}{1} = 20$$

Eftersom 20 er en positiv konstant betyder det at: $f(n) = \Theta(n^2)$.

Sætning 2: Hvis

$$\frac{f(n)}{g(n)} \rightarrow 0 \quad \text{når } n \rightarrow \infty$$

så gælder

$$f(n) = o(g(n))$$

Det vil sige, at hvis f divideret med g går mod nul, betyder det at f bliver relativt forsvindende lille i forhold til g , altså vokser f strengt langsommere end g .

Eksempel:

$$f(n) = 20n^2 + 17n + 312$$

$$g(n) = n^3$$

$$\frac{f(n)}{g(n)} = \frac{20n^2 + 17n + 312}{n^3} = \frac{20}{n} + \frac{17}{n^2} + \frac{312}{n^3} \xrightarrow{n \rightarrow \infty} \frac{0}{1} = 0$$

Eftersom forholdet går mod 0 betyder det at: $f(n) = o(n^3)$.

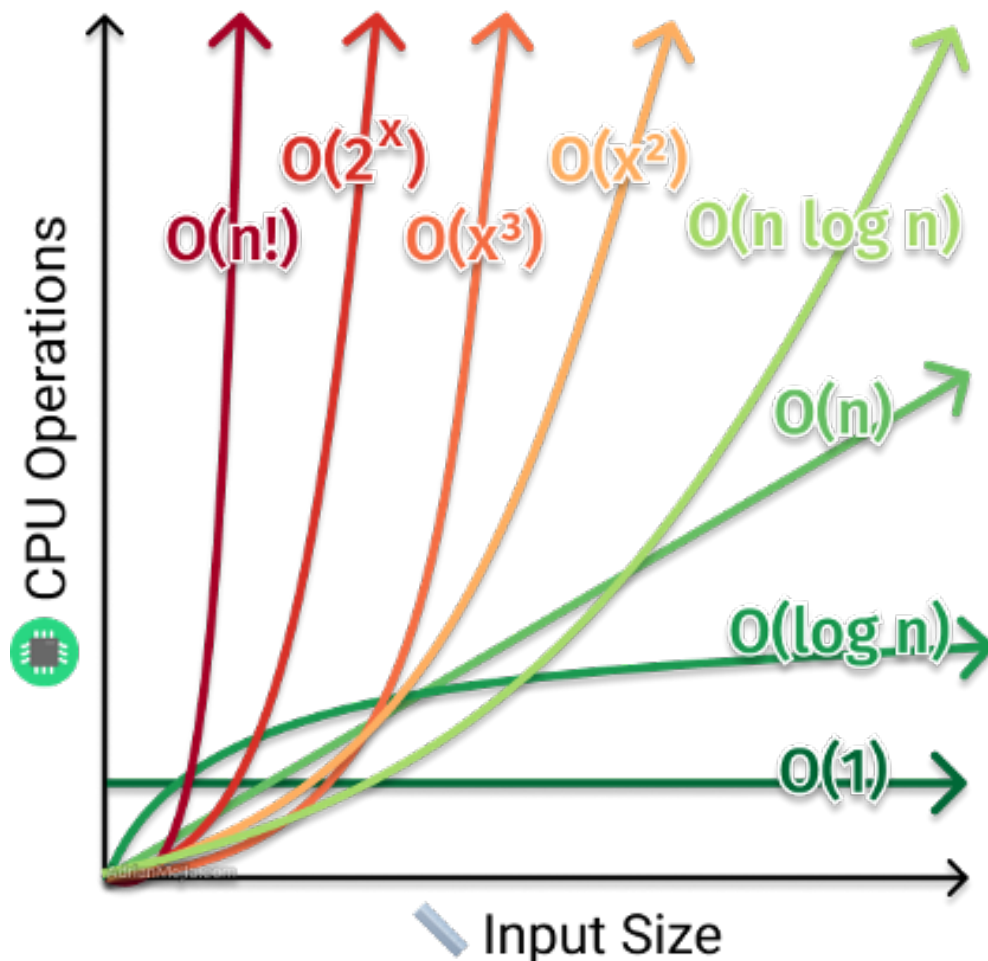
4.7 Vækststrategier

Voksehastigheder kan opstilles fra hurtigst til langsomst som følgende:

$$1, \quad \log n, \quad \sqrt{n}, \quad n, \quad n \log n, \\ n\sqrt{n}, \quad n^2, \quad n^3, \quad n^{10}, \quad 2^n$$

Figur 7: Voksehastigheder ordnet fra hurtigst til langsomst.

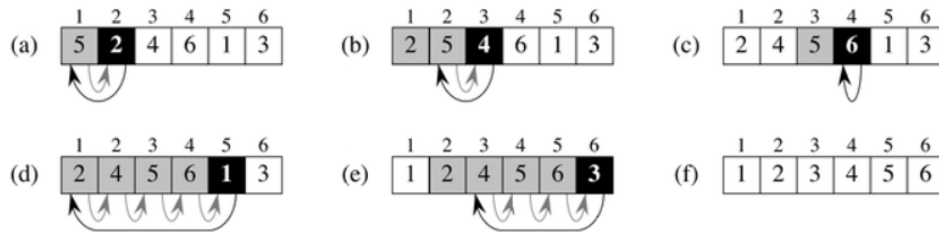
Visualisering af forskellige køretider:



Figur 8: Visualisering af forskellige køretider.

4.8 Invarianter

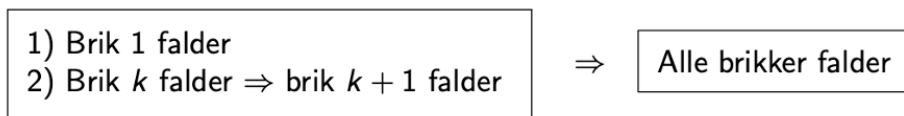
En invariant er et forhold som vedligeholdes af algoritmen i løbet af dens udførelse. Et eksempel på en invariant kan være i insertionsort, hvor alt til venstre for den nuværende key (sorte felt) skal være sorteret.



Figur 9: Invarianten i insertionsort: alt til venstre for den nuværende key er altid sorteret.

Ofte udgør invarianten selve kerneidéen af algoritmen, og kan også bruges som belæg for at algoritmen virker.

En invariant skal overholdes efter hvert skridt (iteration af løkke) udført af algoritmen. Dette kan bevises ved hjælp af induktion:



Figur 10: Induktionsprincippet: holder invarianten i basistilfældet, og bevarer hvert skridt den, så gælder den gennem hele kørslen.

4.9 Analyse af løkker i hånden

Man kan som tommelfingerregel analysere en algoritmes køretid på hvor mange nestede loops de anvender:

```

ALGORITME1( $n$ )
   $i = 1$ 
  while  $i \leq n$ 
     $i = i + 2$ 
  
```

Figur 11: Her har vi ét loop, så køretiden er $\Theta(n)$.

```

ALGORITME2( $n$ )
   $s = 0$ 
  for  $i = 1$  to  $n$ 
    for  $j = i$  downto 1
       $s = s + 1$ 
  
```

Figur 12: Her har vi to nestede loops, så køretiden er $\Theta(n^2)$.

```

ALGORITME3( $n$ )
   $s = 0$ 
  for  $i = 1$  to  $n$ 
    for  $j = i$  to  $n$ 
      for  $k = i$  to  $j$ 
         $s = s + 1$ 

```

Figur 13: Her har vi tre nestede loops, så køretiden er $\Theta(n^3)$.

4.10 Eksamenstips og faldgruber

Noter tilføjes.

5 Rekursionsligninger

5.1 Divide and conquer

Divide and conquer er en algoritme-udviklingsmetode, som er rekursiv. Filosofien bag divide and conquer går ud på:

- Opdel problemet i mindre delproblemer.
- Løs delproblemerne ved rekursion. Kalder sig selv med det mindre delproblem som input.
- Konstruer en løsning til problemet ud fra løsningen af delproblemerne.

Ligesom enhver anden rekursiv algoritme, skal en divide and conquer algoritme også have et basistilfælde, så den ikke kører uendeligt. For divide and conquer betyder det at et problem af mindste størrelse løses direkte uden at benytte rekursion.

En generel struktur af en divide and conquer algoritme kunne se sådan her ud:

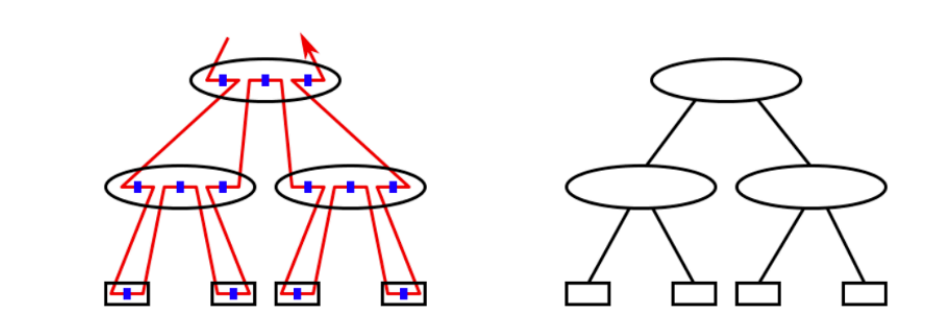
```

If basistilfælde
  Lokalt arbejde (løs problem af basisstørrelse)
Else
  Lokalt arbejde (f.eks. byg et eller flere delproblemer)
  Rekursivt kald
  Lokalt arbejde (f.eks. udnyt svar til at bygge næste delproblem)
  Rekursivt kald
  Lokalt arbejde (løs hovedproblem ud fra svar på delproblemer)

```

Figur 14: Generel struktur af en divide and conquer algoritme.

Arbejdsflowet for en divide and conquer algoritme kan vises således:



Figur 15: Arbejdsflow for en divide and conquer algoritme.

Hvor:

- En knude (oval, firkant) er ét kald af algoritmen.
- En blå prik er lokalt arbejde udført.
- En rød linje er selve det rekursive kald.

5.1.1 Eksempler på divide and conquer algoritmer

Mergesort:

1. Del input op i to dele X og Y .
2. Sorter hver del for sig med rekursion.
3. Merge de to sorterede dele til én sorteret del.

Basistilfælde: $n \leq 1$.

Quicksort:

1. Del input op i to dele X og Y så $X \leq Y$.
2. Sorter hver del for sig.
3. Returner X efterfulgt af Y .

Basistilfælde: $n \leq 1$.

5.2 Rekursionsligning

Køretiden for en rekursiv algoritme kan beskrives ved en rekursionsligning:

$$T(n) = \begin{cases} a \cdot T(n/b) + \Theta(f(n)) & \text{hvis } n > 1 \\ \Theta(1) & \text{hvis } n \leq 1 \end{cases}$$

Figur 16: Generel form af en rekursionsligning.

Hvor:

- a = Hvor mange rekursive kald algoritmen laver.
- b = Hvor meget mindre hvert delproblem bliver (f.eks. halveres $\rightarrow b = 2$).
- $f(n)$ = Hvor meget lokalt arbejde algoritmen laver.
- $\Theta(1)$ = Basistilfældet.

Ofte undlader man at skrive basistilfældet med, fordi det altid er det samme.

Eksempel (Mergesort):

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Mergesort laver 2 rekursive kald: $a = 2$, og hvert kald halverer inputtet: $b = 2$. Til sidst laver den $\Theta(n)$ lokalt arbejde (når der flettes sammen igen).

5.3 Rekursionstræ-metoden

Rekursionstræsmetoden er en opskrift på, hvordan man finder den samlede køretid $T(n)$ ud fra et rekursionstræ.

Trin 1: Annotér hver knude i træet.

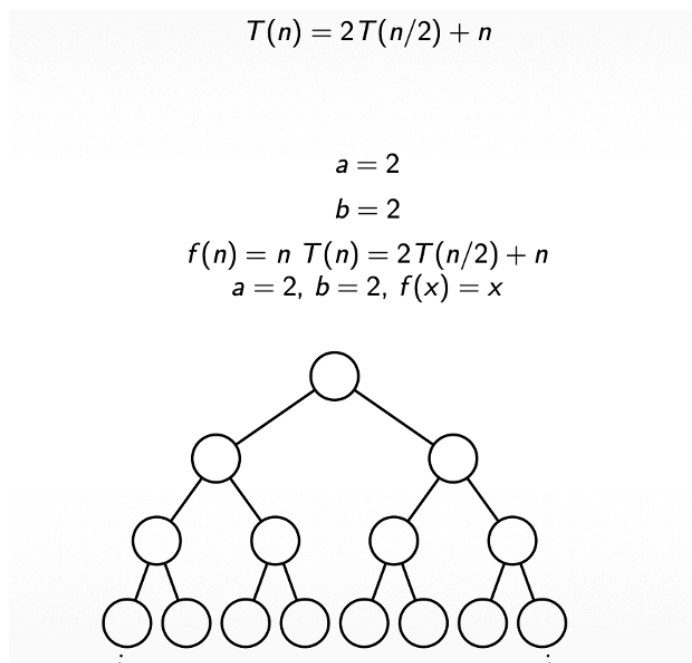
For hver knude (hvert kald), skriv:

- Input-størrelse $(n, \frac{n}{2}, \frac{n}{4})$.
- Arbejdet udført i den knude $(f(n), f(\frac{n}{2}))$.

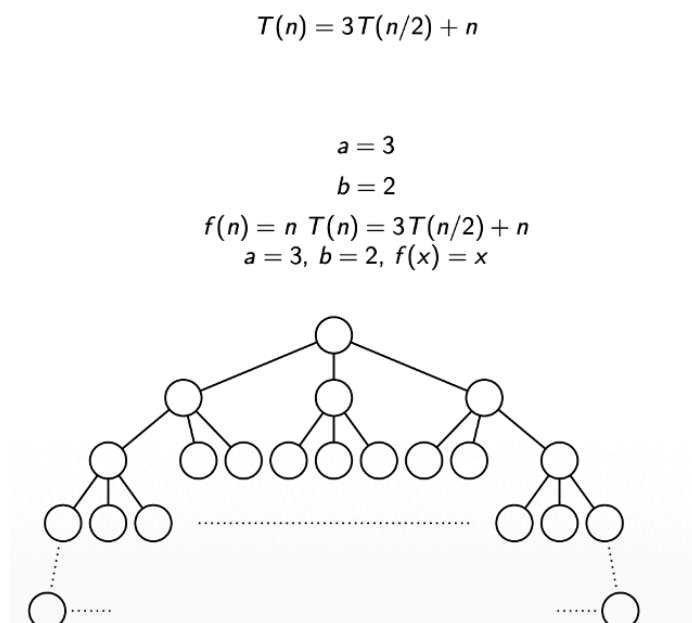
Trin 2: Find den samlede køretid.

Det gøres i tre skridt:

1. Find træets højde.
2. Sum arbejdet i hvert lag for sig.
3. Sum alle lagene sammen.



Figur 17: Eksempel: $T(n) = 2T(\frac{n}{2}) + n$.



Figur 18: Eksempel 2: $T(n) = 3T(\frac{n}{2}) + n$.

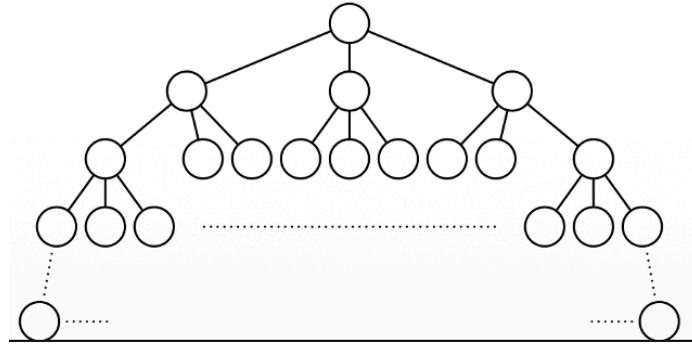
$$T(n) = 3T(n/4) + n^2$$

$$a = 3$$

$$b = 4$$

$$f(n) = n^2 \quad T(n) = 3T(n/4) + n^2$$

$$a = 3, b = 4, f(x) = x^2$$



Figur 19: Eksempel 3: $T(n) = 3T(\frac{n}{4}) + n^2$.

Når man regner rekursionstræmetoden bruger man den geometriske sum:

$$1 + c + c^2 + c^3 + \dots + c^k = \frac{c^{k+1} - 1}{c - 1} = \Theta(c^k)$$

Læg mærke til at summen er $\Theta(c^k)$. Det vil sige at den vokser asymptotisk lige så hurtigt som det største led i summen, her c^k .

Eftersom vi benytter asymptotisk notation, er det irrelevant hvilken base vores logaritme har, da de alle er en konstant faktor fra hinanden. Derfor:

$$\log_b x = \Theta(\log_c x)$$

5.4 Master-sætningen (CLRS 4. udgave)

Master theorem er en “genvejs-tabel” til at finde løsningen på en rekursionsligning der står på formen:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

På den måde kan man undgå at skulle tegne rekursionstræet og summere lagene.

$\alpha = \log_b a$ er det punkt, hvor $f(n)$ “matcher” rekursionens egen vækst. n^α er den vækstrate du ville få hvis algoritmen ikke lavede noget lokalt arbejde overhovedet. Det fungerer som en neutral baseline som $f(n)$ bliver sammenlignet med.

Der er 3 forskellige cases man kan være i, hvorfra man kan aflæse sit svar direkte:

Case 1: $f(n)$ er polynomielt mindre end n^α .

- Formelt: $f(n) = O(n^{\alpha-\varepsilon})$ for et $\varepsilon > 0$.
- $T(n) = \Theta(n^\alpha)$.
- Det lokale arbejde er ubetydeligt. Rekursionen selv dominerer køretiden.

Case 2: $f(n)$ er (cirka) lige stor som n^α .

- Formelt: $f(n) = \Theta(n^\alpha \log^k n)$ for et $k \geq 0$.
- $T(n) = \Theta(n^\alpha \log^{k+1} n)$.
- De to vækstrater matcher hinanden og resultatet bliver det samme, men med en ekstra log-faktor.

Case 3: $f(n)$ er polynomielt større end n^α .

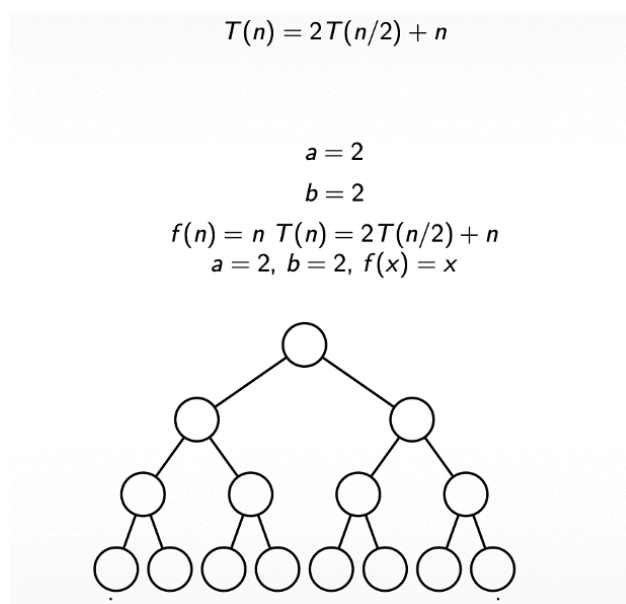
- Formelt: $f(n) = \Omega(n^{\alpha+\varepsilon})$ for et $\varepsilon > 0$.
- $T(n) = \Theta(f(n))$.
- Det lokale arbejde dominerer så meget, at hele køretiden bare bliver lig med arbejdet ved roden.
- Der er en ekstra betingelse ved case 3. Det skal også gælde at: $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$, for en konstant $c < 1$.
- Det sikrer at $f(n)$ ikke hopper vildt op og ned og vokser jævnt. I praksis er denne betingelse næsten altid opfyldt.

Kort sagt: case afgøres af forholdet mellem voksehastighederne for $f(n)$ og for $n^\alpha \log^k n$ hvor $k \geq 0$.

- Er $f(n)$ mindre med mindst en faktor n^ε har vi case 1.
- Er de ens har vi case 2.
- Er $f(n)$ større med mindst en faktor n^ε har vi case 3.

5.4.1 Master theorem anvendt på de tre Mergesort-eksempler

Eksempel 1:



Figur 20: $T(n) = 2T\left(\frac{n}{2}\right) + n$.

Master theorem:

- ▶ $a = 2$
- ▶ $b = 2$
- ▶ $f(n) = n$
- ▶ $\alpha = \log_2 2 = 1$
- ▶ $f(n) = n = \Theta(n^1) = \Theta(n^1(\log n)^0) = \Theta(n^\alpha(\log n)^0)$ [dvs. $k = 0$]

Så vi er i case 2, så der gælder $T(n) = \Theta(n^\alpha(\log n)^{0+1}) = \Theta(n \log n)$.

Figur 21: Master theorem på eksempel 1.

Eksempel 2:

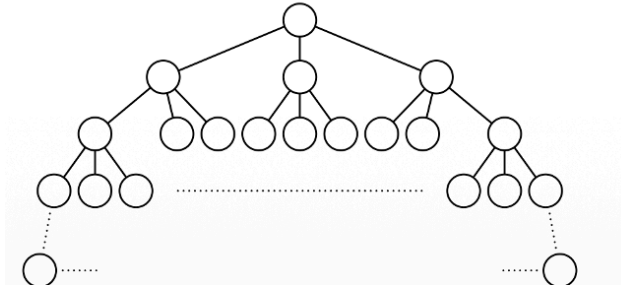
$$T(n) = 3T(n/2) + n$$

$$a = 3$$

$$b = 2$$

$$f(n) = n \quad T(n) = 3T(n/2) + n$$

$$a = 3, b = 2, f(x) = x$$



Figur 22: $T(n) = 3T(\frac{n}{2}) + n$.

Master theorem:

- ▶ $a = 3$
- ▶ $b = 2$
- ▶ $f(n) = n$
- ▶ $\alpha = \log_2 3 = \ln(3)/\ln(2) = 1.5849 \dots$
- ▶ $f(n) = n = O(n^{1.5849 \dots - \epsilon}) = O(n^{\alpha - \epsilon})$ for f.eks. $\epsilon = 0.1$.

Så vi er i case 1, så der gælder $T(n) = \Theta(n^\alpha) = \Theta(n^{1.5849 \dots})$.

Figur 23: Master theorem på eksempel 2.

Eksempel 3:

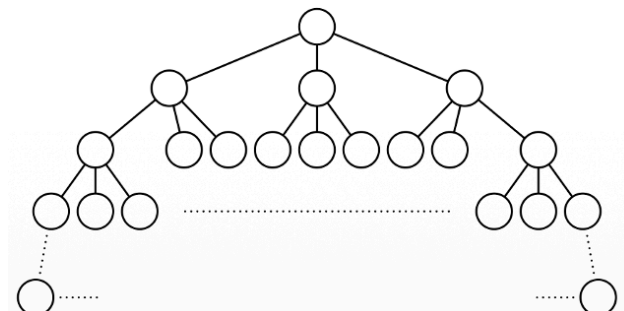
$$T(n) = 3T(n/4) + n^2$$

$$a = 3$$

$$b = 4$$

$$f(n) = n^2 \quad T(n) = 3T(n/4) + n^2$$

$$a = 3, b = 4, f(x) = x^2$$



Figur 24: $T(n) = 3T(\frac{n}{4}) + n^2$.

Master theorem:

- ▶ $a = 3$
- ▶ $b = 4$
- ▶ $f(n) = n^2$
- ▶ $\alpha = \log_4 3 = \ln(3)/\ln(4) = 0.7924 \dots$
- ▶ $f(n) = n^2 = \Omega(n^{0.7924 \dots + \epsilon}) = \Omega(n^{\alpha + \epsilon})$ for f.eks. $\epsilon = 0.1$.

Så vi er i case 3, så der gælder $T(n) = \Theta(f(n)) = \Theta(n^2)$.

I case 3 skal vi dog også checke den ekstra betingelse: Med $c = 3/16 < 1$ og $n_0 = 1$ opfyldes at $a \cdot f(n/b) = 3(n/4)^2 = 3/16 \cdot n^2 \leq c \cdot f(n) = c \cdot n^2$ for alle $n \geq n_0$.

Figur 25: Master theorem på eksempel 3.

5.4.2 Når master theorem ikke kan anvendes

Der er nogle rekursionsligninger hvor master theorem ikke kan bruges. Et eksempel kan være:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(2\frac{n}{3}\right) + n$$

Master theorem kræver at alle rekursive kald $\frac{n}{b}$ er lige store, hvilket de i det her tilfælde ikke er. Dette er også et asymmetrisk træ. I sådanne tilfælde bliver man nødt til at benytte sig af rekursionstræmetoden.

5.4.3 Ulige input (afrunding)

Vi kan komme ud for et scenarie, hvor inputtet n er et ulige tal. Dette giver os to rekursive kald hvor inputtet ikke kan deles lige op. For at komme uden om det deler vi i en halvdel der er rundet op $\lceil \frac{n}{2} \rceil$, og en anden halvdel der er rundet ned $\lfloor \frac{n}{2} \rfloor$.

Som eksempel går Mergesort fra:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n$$

Til:

$$T(n) = T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$$

Dette gør ingen forskel for beregningen af køretiden.

5.5 Matrixmultiplikation

En matrix er en firkant af tal:

$$\begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 7 \\ 9 & 1 & 1 \end{bmatrix}$$

Figur 26: En matrix.

Alle matricer er $n \times n$ kvadratiske. Den ovenstående er 3×3 .

Addition for matricer sker som følgende:

$$\begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 7 \\ 9 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 3 & 2 & 1 \\ 4 & 3 & 2 \\ 5 & 4 & 3 \end{bmatrix} = \begin{bmatrix} 1+3 & 6+2 & 4+1 \\ 2+4 & 5+3 & 7+2 \\ 9+5 & 1+4 & 1+3 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 5 \\ 6 & 8 & 9 \\ 14 & 5 & 4 \end{bmatrix}$$

Figur 27: Addition af to matricer.

Addition af matricer har køretid: $\Theta(n^2)$.

Gange for matricer sker som følgende:

$$\begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 7 \\ 9 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 & 2 & 1 \\ 4 & 3 & 2 \\ 5 & 4 & 3 \end{bmatrix} = \begin{bmatrix} ? & ? & ? \\ ? & ? & 33 \\ ? & ? & ? \end{bmatrix}$$

$$33 = 2 \cdot 1 + 5 \cdot 2 + 7 \cdot 3$$

$$\begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 7 \\ 9 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 & 2 & 1 \\ 4 & 3 & 2 \\ 5 & 4 & 3 \end{bmatrix} = \begin{bmatrix} ? & ? & ? \\ ? & ? & 33 \\ ? & 25 & ? \end{bmatrix}$$

$$25 = 9 \cdot 2 + 1 \cdot 3 + 1 \cdot 4$$

Figur 28: Multiplikation af to matricer.

Multiplikation af matricer har køretid: $\Theta(n^3)$ — kan det gøres bedre? Ikke med en rekursiv algoritme. Den giver samme køretid, hvilket master theorem afspejler:

$$T(n) = 8T(n/2) + n^2$$

Master theorem:

- ▶ $\alpha = \log_b(a) = \log_2(8) = 3$
- ▶ $f(n) = n^2$

$$n^2 = O(n^{\alpha-0.1}) \Rightarrow \text{Case 1}$$

$$T(n) = \Theta(n^\alpha) = \Theta(n^3)$$

Figur 29: Master theorem på den rekursive matrixmultiplikation: $\Theta(n^3)$.

5.5.1 Strassens algoritme

Vi kan med fordel bruge Strassen algoritmen for at få en bedre køretid.

Først beregner vi 10 hjælpematricer:

$S_1 = B_{12} - B_{22}$	$S_6 = B_{11} + B_{22}$
$S_2 = A_{11} + A_{12}$	$S_7 = A_{12} - A_{22}$
$S_3 = A_{21} + A_{22}$	$S_8 = B_{21} + B_{22}$
$S_4 = B_{21} - B_{11}$	$S_9 = A_{11} - A_{21}$
$S_5 = A_{11} + A_{22}$	$S_{10} = B_{11} + B_{12}$

Figur 30: De 10 hjælpematricer.

De bruges til at finde kun 7 produkter frem for de før 8. Det er vores rekursive kald:

$$P_1 = A_{11} \cdot S_1$$

$$P_2 = S_2 \cdot B_{22}$$

$$P_3 = S_3 \cdot B_{11}$$

$$P_4 = A_{22} \cdot S_4$$

$$P_5 = S_5 \cdot S_6$$

$$P_6 = S_7 \cdot S_8$$

$$P_7 = S_9 \cdot S_{10}$$

Figur 31: De 7 produkter (de rekursive kald).

Til sidst kan de fire værdier findes i 2×2 matrixen:

$$\begin{aligned}
P_5 + P_4 - P_2 + P_6 &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} \\
P_1 + P_2 &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\
P_3 + P_4 &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\
P_5 + P_1 - P_3 - P_7 &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22}
\end{aligned}$$

Figur 32: De fire værdier i resultatmatrixen.

$$\begin{aligned}
A_{11} \cdot B_{11} + A_{12} \cdot B_{21} &= C_{11} \\
A_{11} \cdot B_{12} + A_{12} \cdot B_{22} &= C_{12} \\
A_{21} \cdot B_{11} + A_{22} \cdot B_{21} &= C_{21} \\
A_{21} \cdot B_{12} + A_{22} \cdot B_{22} &= C_{22}
\end{aligned}$$

Figur 33: Resultatmatrixen.

Ved hjælp af master theorem kan vi nu vurdere den nye køretid når Strassen algoritmen benyttes:

$$T(n) = 7T(n/2) + n^2$$

Master theorem:

- ▶ $\alpha = \log_b(a) = \log_2(7) = 2.80735 \dots$
- ▶ $f(n) = n^2$

$$n^2 = O(n^{\alpha-0.1}) \Rightarrow \text{Case 1}$$

$$T(n) = \Theta(n^\alpha) = O(n^{2.81})$$

Figur 34: Master theorem på Strassens algoritme.

5.6 Eksamenstips og faldgruber

Noter tilføjes.

6 Grafgennemløb

6.1 Repræsentationer

Noter tilføjes (nabolister vs. nabomatrix).

6.2 Bredde-først søgning (BFS)

Noter tilføjes (d-værdier, besøgsrækkefølge).

6.3 Dybde-først søgning (DFS)

Noter tilføjes (discovery-/finish-tider; kantklassifikation: træ-/tilbage-/frem-/krydskanter).

6.4 Topologisk sortering

Noter tilføjes.

6.5 Eksamenstips og faldgruber

Noter tilføjes (konvention om alfabetisk ordnede nabolister).

7 Sortering

7.1 Sammenligningsbaserede sorteringer

Noter tilføjes (quicksort + PARTITION, merge sort, heapsort).

7.2 Lineærtids-sorteringer

Noter tilføjes (counting sort, radix sort, bucket sort).

7.3 Sammenligningstabel

Noter tilføjes (bedste/gennemsnitlige/værste tilfælde, in-place, stabil).

7.4 Eksamenstips og faldgruber

Noter tilføjes.

8 Minimum udspændende træ

8.1 Snit-egenskaben (cut property)

Noter tilføjes.

8.2 Kruskals algoritme

Noter tilføjes (hvilken kant kommer med næst; #sammenhængskomponenter undervejs).

8.3 Prims algoritme

Noter tilføjes.

8.4 Eksamenstips og faldgruber

Noter tilføjes.

9 Disjunkte mængder (Union-Find)

9.1 Operationer

Noter tilføjes (MAKE-SET, UNION, FIND-SET).

9.2 Union by rank

Noter tilføjes.

9.3 Path compression (stikomprimering)

Noter tilføjes.

9.4 Eksamenstips og faldgruber

Noter tilføjes (tegne skoven efter en sekvens af operationer).

10 Grådige algoritmer og Huffman-kodning

10.1 Det grådige princip

Noter tilføjes (grådigt valg, ombytningsargument).

10.2 Huffman-kodning

Noter tilføjes (opbygning af træet, kodeordslængder, samlet antal bits).

10.3 Eksamenstips og faldgruber

Noter tilføjes.

11 Søgetræer

11.1 Binære søgetræer

Noter tilføjes (søgning, indsættelse, sletning, gennemløb).

11.2 Rød-sorter træer

Noter tilføjes (de fem egenskaber; tjek af gyldig farvning).

11.3 Indsættelse og rotationer

Noter tilføjes.

11.4 Udvidede (augmented) træer

Noter tilføjes.

11.5 Eksamenstips og faldgruber

Noter tilføjes.

12 Heaps og prioritetskøer

12.1 Heap-egenskaben og array-repræsentation

Noter tilføjes (forælder-/barn-indeks, min- vs. max-heap).

12.2 Opbygning af en heap

Noter tilføjes (BUILD-HEAP, HEAPIFY).

12.3 Operationer

Noter tilføjes (indsæt, extract-min/max, decrease-key).

12.4 Eksamenstips og faldgruber

Noter tilføjes.

13 Dynamisk programmering

13.1 Principper

Noter tilføjes (optimal delstruktur, overlappende delproblemer, memoisering vs. bottom-up).

13.2 Gennemregnede eksempler

Noter tilføjes.

13.3 Køretids- og pladsanalyse

Noter tilføjes (aflæse kompleksitet ud fra en rekursionsligning; reducere pladsforbrug).

13.4 Eksamenstips og faldgruber

Noter tilføjes.

14 Hashing og ordbøger (dictionaries)

14.1 Kædning (chaining)

Noter tilføjes.

14.2 Åben adressering

Noter tilføjes (linear probing, double hashing).

14.3 Fyldningsgrad (load factor) og ydeevne

Noter tilføjes.

14.4 Eksamenstips og faldgruber

Noter tilføjes (rekonstruere indsættelsesrækkefølge / mulige hashværdier).

15 Korteste veje

15.1 Relaksering (relaxation)

Noter tilføjes.

15.2 Dijkstras algoritme

Noter tilføjes (ingen negative kanter).

15.3 Bellman-Ford

Noter tilføjes (håndterer negative kanter; detekterer negative cykler).

15.4 Korteste veje i en DAG

Noter tilføjes.

15.5 Hvilken algoritme skal bruges

Noter tilføjes (valg ud fra negative kanter / cykler / DAG).

15.6 Eksamenstips og faldgruber

Noter tilføjes.